Proyecto No. 2: Simplificación de gramáticas y algoritmo CYK

Teoría de la Computación — Universidad del Valle de Guatemala

Samuel Robledo — 241282 19 de octubre de 2026

https://github.com/samuelrobledo52/Proyecto2-TC

1. Diseño

El programa lee una gramática desde un archivo, la convierte a CNF y valida frases con CYK. Está dividido en cinco módulos:

- `gramatica.py`: guarda las producciones como `variable → lista de tuplas`; la tupla vacía es ε. Lo que nunca aparece como cabeza es terminal.
- `cnf.py`: una función por paso de simplificación.
- `cyk.py`: llena la tabla y separa la frase en tokens, aunque venga sin espacios.
- `arbol.py`: arma el parse tree desde la tabla y lo dibuja con Graphviz.
- `main.py`: recibe la gramática y las frases y guarda todo en `resultados.txt`.

2. Conversión a CNF

Se sigue el orden de Hopcroft, porque cada paso puede dejar casos que el siguiente limpia:

1. Si el inicial es anulable y aparece en un cuerpo, se agrega $S_0 \to S$.
2. Se eliminan las producciones ε generando las combinaciones sin anulables.
3. Se eliminan las unitarias con los pares (A, B) tales que $A \Rightarrow^* B$.
4. Se quitan los símbolos que no generan y luego los que no son alcanzables.
5. Los terminales en cuerpos de largo 2 o más se cambian por variables ($L \to (, R \to)$, $P \to +$, $M \to *$).
6. Los cuerpos largos se parten en pares con variables nuevas $C_1, C_2, \ldots$

Con `1.txt` se obtiene la misma gramática de `1-cnf.txt`; solo cambian los nombres (Z, W, V pasan a ser C_1, C_2, C_3):

$$E \to TX \mid FY \mid LC_1 \mid id \quad X \to PC_2 \mid PT \quad T \to FY \mid LC_1 \mid id$$
$$Y \to MC_3 \mid MF \quad F \to LC_1 \mid id \quad C_1 \to ER,\ C_2 \to TX,\ C_3 \to FY$$

La gramática del enunciado ya está en CNF, así que ningún paso la cambia.

3. Algoritmo CYK

$T[i][j]$ contiene las variables que generan $w_i \cdots w_j$. Cada celda se calcula con celdas de subcadenas más cortas:

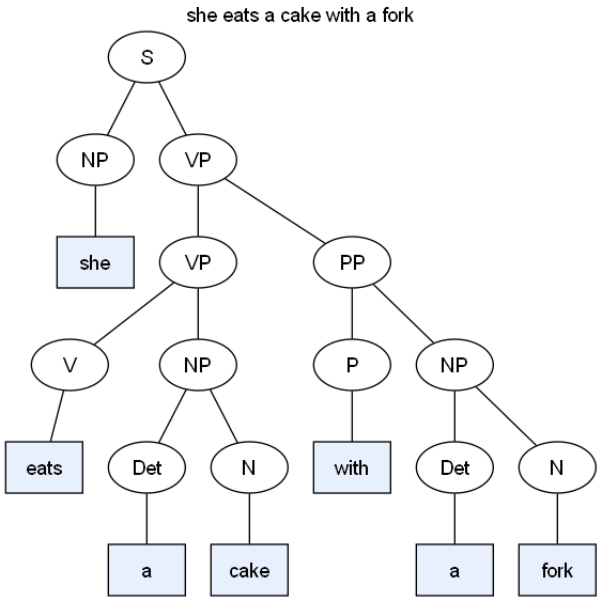
$$T[i][i] = \{A \mid A \to w_i\}, \qquad T[i][j] = \{A \mid A \to BC,\ B \in T[i][k],\ C \in T[k+1][j],\ i \le k < j\}$$

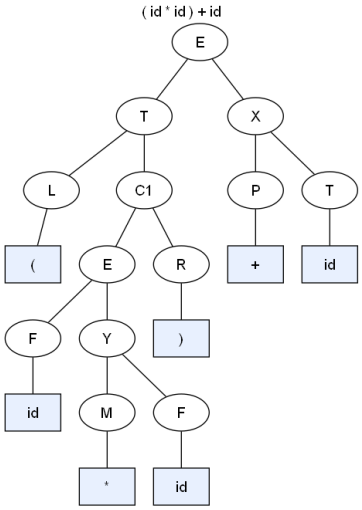
La frase se acepta si $S \in T[1][n]$, con costo $O(n^3|G|)$. Para el árbol, cada celda guarda también de dónde salió cada variable (k, B, C). El tiempo se mide solo sobre el llenado de la tabla.

4. Ejemplos y pruebas

Frase	Res.	ms	Expresión	Res.	ms
She eats a cake with a fork	SÍ	0.032	`(id*id)+id`	SÍ	0.035
The cat drinks the beer	SÍ	0.035	`id+id*id`	SÍ	0.027
she cuts the meat with a knife in the oven	SÍ	0.085	`id*(id+id)*id`	SÍ	0.047
he cooks	SÍ	0.013	`((id))`	SÍ	0.030
she eats cake	NO	0.007	`id+`	NO	0.010
the dog eats with a fork the soup	NO	0.026	`(id*id`	NO	0.011
she eats a pizza	NO	0.008	`()`	NO	0.003

She eats cake falla porque NP necesita determinante, *the dog eats with a fork the soup* porque no existe $VP \to VP\ NP$ y *pizza* no es terminal.





Para revisar la conversión, `pruebas/equivalencia.py` compara la gramática original (con un parser de Earley) contra CYK sobre la CNF. Se probaron 3 000 gramáticas aleatorias y todas las cadenas cortas de `1.txt` y del enunciado, sin diferencias.

5. Discusión

Obstáculos. El orden de los pasos importaba: si se quitaban las unitarias antes que ε, volvían a aparecer. En `1.txt` ya existe T, así que los nombres nuevos se validan para no chocar. Agregar siempre S_0 duplicaba las producciones de E, por eso solo se agrega cuando hace falta. Las expresiones vienen sin espacios y las frases con mayúsculas o punto, y eso se resolvió en la tokenización. Los tiempos son muy pequeños y cambian entre corridas.

Recomendaciones. Guardar todas las derivaciones por celda permitiría ver todos los árboles de una frase ambigua. La gramática del enunciado no tiene $NP \to NP\ PP$, por lo que *the cat with a fork eats* se rechaza. Con muchos anulables en un mismo cuerpo, la CNF crece bastante.

Se utilizó Claude como apoyo en la implementación y la documentación. Todo el contenido fue revisado, probado y validado por mí.